

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра автоматики и процессов управления

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Программирование»
Тема: Динамические массивы. Умножение и возведение в степень
матриц с вводом размерностей с клавиатуры

Студент гр. 6391

Рудой И.В.

Преподаватель

Писарев А.С.

Санкт-Петербург

2026

Цель работы (задание)

Преобразовать программу из задания №7 таким образом, чтобы размерности матриц могли вводиться с клавиатуры.

Примечание: Задача связана с использованием динамических массивов. При решении необходимо определить порядок пересчета двумерных индексов элементов в линейный индекс, определяющий позицию элемента в динамическом массиве.

Программа должна позволять выполнять две операции над матрицами: умножение двух матриц произвольных совместимых размеров и возведение квадратной матрицы в целую положительную степень.

Алгоритм решения

Так как размерности матриц заранее неизвестны и вводятся пользователем во время работы программы, статические двумерные массивы использовать нельзя. Вместо них для каждой матрицы выделяется один линейный динамический массив типа `double` (оператор `new[]`), а его память освобождается оператором `delete[]`.

Порядок пересчёта индексов. Матрица хранится в линейном массиве по строкам. Элемент (i, j) матрицы, имеющей W столбцов, находится в линейном массиве по индексу $i * W + j$. Таким образом:

- Матрица A размером $M \times N$: $A[i][j] \rightarrow A[i * N + j]$ (ширина N), всего $M * N$ элементов;
- Матрица B размером $N \times K$: $B[i][j] \rightarrow B[i * K + j]$ (ширина K), всего $N * K$ элементов;
- Результат $C = A \cdot B$ размером $M \times K$: $C[i][j] \rightarrow C[i * K + j]$ (ширина K), всего $M * K$ элементов.

Алгоритм работы программы:

1. С клавиатуры вводятся размеры M и N матрицы A , выделяется память под $M * N$ элементов, затем элементы матрицы вводятся построчно.
2. Пользователь выбирает операцию: 1 – умножение, 2 – возведение в степень.

3. Умножение (функция multiply). Вводится число столбцов K матрицы B ; число строк B равно числу столбцов A (N), поэтому условие согласованности размеров выполняется автоматически. Выделяется память под B ($N * K$) и под результат C ($M * K$). Матрица C обнуляется, затем для каждого элемента вычисляется сумма $C[i][j] += A[i][k] * B[k][j]$ по k от 0 до $N - 1$.
4. Возведение в степень (функция power). Операция возможна только для квадратной матрицы ($M = N$); иначе выводится сообщение об ошибке. Степень n должна быть целым положительным числом, иначе выводится сообщение об ошибке. Результат сначала приравнивается к A (степень 1), затем $n - 1$ раз умножается на A с помощью функции multiply; промежуточный результат хранится во временном массиве temp.
5. Результат C выводится построчно с помощью `printf("%10.2f");`; после этого вся выделенная динамическая память освобождается.

Текст программы

```
#include <iostream>
#include <cstdio>

using namespace std;

//Пересчёт индексов: элемент (i, j) лежит в линейном массиве по адресу i *
//ширина + j, где ширина – число столбцов матрицы.

//Умножение двух матриц: C = A * B
void multiply(double* A, double* B, double* C, int M, int N, int K)
{
    //Обнуляем C перед накоплением
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < K; j++) {
            C[i * K + j] = 0.0;
        }
    }

    //Умножение матриц
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < K; j++) {
            for (int k = 0; k < N; k++) {
                C[i * K + j] += A[i * N + k] * B[k * K + j];
            }
        }
    }
}
```

```

}

//Возведение матрицы A в степень n: result = A в n степени
//Работает для n >= 1 (n – положительное целое)
void power(double* A, int n, double* result, int S)
{
    //n >= 1: начинаем с A в 1
    for (int i = 0; i < S; i++) {
        for (int j = 0; j < S; j++) {
            result[i * S + j] = A[i * S + j];
        }
    }

    //При n = 1 дальше умножать нечего, возвращаем просто матрицу A
    if (n == 1) {
        return;
    }

    //Для n >= 2: на каждом шаге умножаем текущий результат на A
    double* temp = new double [S*S];
    for (int p = 2; p <= n; p++) {
        multiply(result, A, temp, S, S, S);    // temp = result * A = A^p

        //Копируем temp в result, чтобы использовать на следующем шаге
        for (int i = 0; i < S * S; i++) {
            result[i] = temp[i];
        }
    }
    delete[] temp;
}

int main()
{
    int M, N;
    cout << "Введите размеры матрицы A размерами M на N:" << endl;
    cin >> M >> N;

    cout << "Введите элементы матрицы по строкам:" << endl;
    //Выделяем память под M×N элементов
    double* A = new double[M * N];

    //Ввод построчно: A[i][j] = A[i * N + j], ширина = N
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            cin >> A[i * N + j];
        }
    }

    int choice;
    cout << "Что делать дальше? (1 – умножение, 2 – возведение в степень)" <<
endl;
    cin >> choice;
}

```

```

switch (choice) {
    case 1: {
        int K;
        cout << "Введите количество столбцов матрицы B для умножения:" <<
endl;

        cin >> K;
        //B имеет размер N×K (строки B = столбцы A)
        double* B = new double[N * K];

        //Ввод B: B[i][j] = B[i * K + j], ширина = K
        cout << "Введите элементы матрицы по строкам:" << endl;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < K; j++) {
                cin >> B[i * K + j];
            }
        }

        double* C = new double[M * K];
        multiply(A, B, C, M, N, K);

        //Вывод C построчно, C[i][j] = C[i * K + j]
        cout << "Вывод итоговой матрицы C:" << endl;
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < K; j++) {
                printf("%10.2f", C[i * K + j]);
            }
            printf("\n");
        }

        //Освобождаем память, выделенную в этой ветке
        delete[] B;
        delete[] C;
        break;
    }
    case 2: {
        int n;

        //Для степени матрица должна быть квадратной
        if (M != N) {
            cout << "Матрица не квадратная, возведение в степень
невозможно" << endl;
            break;
        }
        else {
            cout << "Введите степень:" << endl;
            cin >> n;

            //Степень должна быть положительной
            if (n < 1) {
                cout << "Степень должна быть целым положительным числом!"
<< endl;

                break;
            }
            else {

```

```

        //C – размер M×N (то же, что M×M, так как M == N)
        double* C = new double[M * N];
        power(A, n, C, M);
        cout << "Вывод итоговой матрицы C:" << endl;
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < N; j++) {
                printf("%10.2f", C[i * N + j]);
            }
            printf("\n");
        }
        //Освобождаем память, выделенную в этой ветке
        delete[] C;
        break;
    }
}
default:
    cout << "Неверный выбор." << endl;
    break;
}
//А нужна была до конца – освобождаем здесь
delete[] A;
return 0;
}

```

Описание тестов

Для каждого теста указаны входные данные и ожидаемый результат. Данные вводятся в порядке запроса программы: размеры матрицы A (M N), элементы A, номер операции, затем параметры операции (для умножения – K и элементы B, для возведения в степень – n).

Умножение матриц

Тест 1. Умножение прямоугольных матриц 2×3 и 3×2

Вход:

M N: 2 3

A: 1 2 3

4 5 6

Операция: 1, K = 2

B: 3 3

1 0

2 0

Ожидаемый выход:

11.00 3.00

29.00 12.00

Результат имеет размер $M \times K = 2 \times 2$. Например, $C[0][0] = 1 \cdot 3 + 2 \cdot 1 + 3 \cdot 2 = 11$,
 $C[1][1] = 4 \cdot 3 + 5 \cdot 0 + 6 \cdot 0 = 12$.

Тест 2. Умножение матриц 3×2 и 2×3

Вход:

M N: 3 2

A: 1 2

3 4

5 6

Операция: 1, K = 3

B: 1 2 3

4 5 6

Ожидаемый выход:

9.00 12.00 15.00

19.00 26.00 33.00

29.00 40.00 51.00

Результат имеет размер 3×3 и больше исходных матриц. Тест проверяет, что память под C выделяется по размеру $M \times K$, а не по размеру исходных матриц.

Тест 3. Умножение на единичную матрицу, дробные и отрицательные числа

Вход:

M N: 2 2

A: 1.5 2

-1 0.5

Операция: 1, K = 2

B: 1 0

0 1

Ожидаемый выход:

1.50	2.00
-1.00	0.50

При умножении на единичную матрицу результат совпадает с исходной матрицей. Тест проверяет работу с вещественными и отрицательными числами.

Возведение матрицы в степень

Тест 4. Квадрат матрицы 2×2

Вход:

M N: 2 2

A: 1 2

3 4

Операция: 2, n = 2

Ожидаемый выход:

7.00	10.00
15.00	22.00

$A^2 = A \cdot A$: $1 \cdot 1 + 2 \cdot 3 = 7$; $1 \cdot 2 + 2 \cdot 4 = 10$; $3 \cdot 1 + 4 \cdot 3 = 15$; $3 \cdot 2 + 4 \cdot 4 = 22$.

Тест 5. Куб матрицы 3×3

Вход:

M N: 3 3

A: 1 0 2

0 1 0

1 1 1

Операция: 2, n = 3

Ожидаемый выход:

7.00	6.00	10.00
0.00	1.00	0.00
5.00	5.00	7.00

Тест проверяет цикл многократного умножения: $A^2 = [[3,2,4],[0,1,0],[2,2,3]]$, $A^3 = A^2 \cdot A$. Размер матрицы задаётся пользователем (3×3).

Тест 6. Матрица 1×1 , степень 4

Вход:

M N: 1 1

A: 3

Операция: 2, $n = 4$

Ожидаемый выход:

81.00

Граничный случай минимального размера матрицы: $3^4 = 81$. Проверяет корректность динамического выделения памяти для одного элемента.

Тест 7. Некорректная степень ($n = 0$)

Вход:

M N: 2 2

A:

1.0 2.0

3.0 4.0

Операция: 2, $n = 0$

Ожидаемый выход:

Степень должна быть целым положительным числом!

Тест 8. Проверка квадратности: матрица 2×4 ($M \neq N$)

Вход:

M N: 2 4

A:

1.0 2.0 3.0 4.0

5.0 6.0 7.0 8.0

Операция: 2

Ожидаемый выход:

Матрица не квадратная, возведение в степень невозможно

Результат работы

Ниже приведены скриншоты консоли, подтверждающие корректную работу программы на разработанных тестах.

```

Введите размеры матрицы A размерами M на N:
2 3
Введите элементы матрицы по строкам:
1 2 3
      4 5 6
Что делать дальше? (1 - умножение, 2 - возведение в степень)
1
Введите количество столбцов матрицы B для умножения:
2
Введите элементы матрицы по строкам:
3 3
      1 0
      2 0
Вывод итоговой матрицы C:
      11.00      3.00
      29.00      12.00

```

Рисунок 1. Результаты выполнения теста 1

```

Введите размеры матрицы A размерами M на N:
3 2
Введите элементы матрицы по строкам:
1 2
3 4
5 6
Что делать дальше? (1 - умножение, 2 - возведение в степень)
1
Введите количество столбцов матрицы B для умножения:
3
Введите элементы матрицы по строкам:
1 2 3
4 5 6
Вывод итоговой матрицы C:
      9.00      12.00      15.00
     19.00     26.00     33.00
     29.00     40.00     51.00

```

Рисунок 2. Результаты выполнения теста 2

```

Введите размеры матрицы A размерами M на N:
2 2
Введите элементы матрицы по строкам:
1.5  2
      -1  0.5
Что делать дальше? (1 - умножение, 2 - возведение в степень)
1
Введите количество столбцов матрицы B для умножения:
2
Введите элементы матрицы по строкам:
1 0
0 1
Вывод итоговой матрицы C:
      1.50      2.00
     -1.00      0.50

```

Рисунок 3. Результаты выполнения теста 3

```

Введите размеры матрицы A размерами M на N:
2 2
Введите элементы матрицы по строкам:
1 2
3 4
Что делать дальше? (1 - умножение, 2 - возведение в степень)
2
Введите степень:
2
Вывод итоговой матрицы C:
      7.00      10.00
     15.00     22.00

```

Рисунок 4. Результаты выполнения теста 4

```

Введите размеры матрицы A размерами M на N:
3 3
Введите элементы матрицы по строкам:
1 0 2
    0 1 0
    1 1 1
Что делать дальше? (1 - умножение, 2 - возведение в степень)
2
Введите степень:
3
Вывод итоговой матрицы C:
    7.00    6.00    10.00
    0.00    1.00    0.00
    5.00    5.00    7.00

```

Рисунок 5. Результаты выполнения теста 5

```

Введите размеры матрицы A размерами M на N:
1 1
Введите элементы матрицы по строкам:
3
Что делать дальше? (1 - умножение, 2 - возведение в степень)
2
Введите степень:
4
Вывод итоговой матрицы C:
    81.00

```

Рисунок 6. Результаты выполнения теста 6

```

Введите размеры матрицы A размерами M на N:
2 2
Введите элементы матрицы по строкам:
1 2
3 4
Что делать дальше? (1 - умножение, 2 - возведение в степень)
2
Введите степень:
0
Степень должна быть целым положительным числом!

```

Рисунок 7. Результаты выполнения теста 7

```

Введите размеры матрицы A размерами M на N:
2 4
Введите элементы матрицы по строкам:
1.0 2.0 3.0 4.0
5.0 6.0 7.0 8.0
Что делать дальше? (1 - умножение, 2 - возведение в степень)
2
Матрица не квадратная, возведение в степень невозможно

```

Рисунок 8. Результаты выполнения теста 8

Все тесты отработали в соответствии с ожидаемыми результатами. Программа корректно выполняет умножение матриц и возведение матрицы в степень при размерностях, вводимых с клавиатуры.

Вывод

В ходе лабораторной работы была доработана программа на языке C++, выполняющая умножение матриц и возведение квадратной матрицы в целую

положительную степень, таким образом, что размерности матриц вводятся с клавиатуры.

Для хранения матриц использованы динамические одномерные массивы, выделяемые оператором `new[]` и освобождаемые оператором `delete[]`. Двумерные индексы пересчитываются в линейный по формуле $i * W + j$, где W – число столбцов матрицы. Предусмотрены проверки: квадратность матрицы и положительность степени при возведении в степень.

Тестирование подтвердило правильность работы программы на различных наборах данных. В процессе работы закрепились навыки работы с динамической памятью, указателями и представлением многомерных данных в одномерном массиве.